A RESEARCH MONOGRAPH OF THE SERIES ON INTEGRATED AUTONOMOUS INTELLIGENCE

ANTIGRAVITY FOR CORPORATE TAX OPTIMIZATION

Building an Interactive Planning Sandbox for Multi-Jurisdiction and Multi-Industry Groups

AGENTIC FINANCE · TAX TECHNOLOGY · GOVERNED ANALYTICS

"The value of an agentic tax model lies not in producing one optimal structure, but in making assumptions, trade-offs, constraints, and risks visible."

ALEJANDRO REYNOSO

JUDGE BUSINESS SCHOOL · UNIVERSITY OF CAMBRIDGE · JULY 2026

Abstract

This report explains how Antigravity can help a finance or tax professional turn a complex corporate tax planning problem into an interactive, inspectable software application. The case study is a synthetic multinational group operating across several jurisdictions and business lines. Beginning with a short natural-language request, the project evolved through a supervised agentic workflow: the human user supplied tax intent and product judgement, while Antigravity converted that intent into source files, formulas, dashboards, tests, and documentation. The paper follows the structure and pedagogical tone of the companion Antigravity algorithmic trading report, but translates the example from portfolio strategy to corporate tax architecture. It shows how the agentic workflow produced a sandbox for effective tax rates, withholding taxes, thin-capitalization limits, US GILTI exposure, and OECD Pillar Two top-up taxes. The appendices record the core mathematical formulations, JavaScript implementation patterns, and prompting trail so that developers can inspect, reproduce, and extend the model.

Keywords: Antigravity; corporate tax optimization; transfer pricing; withholding tax; GILTI; OECD Pillar Two; tax technology; agentic development.

Central proposition

Antigravity is most valuable in corporate tax planning when it is used as a supervised development environment: the tax professional supplies doctrine, constraints, assumptions, and judgement, while the agent turns those inputs into a transparent model whose calculations, interfaces, risks, and documentation can be inspected and revised.

Series note. This monograph forms part of the Integrated Autonomous Intelligence collection. It examines how agentic software development can translate specialist financial reasoning into governed analytical tools.

Research status. The Aether Global Group case is synthetic and pedagogical. The application demonstrates a reproducible modeling workflow; it does not provide legal or tax advice.

Executive Orientation

Corporate tax planning combines legal doctrine, financial modeling, organizational design, and software implementation. The central challenge is not merely to calculate tax. It is to make the interaction between entity residence, intercompany pricing, financing, withholding taxes, controlled foreign corporation rules, and global minimum tax regimes understandable to decision makers.

This monograph presents Antigravity as a bridge between domain expertise and implementation. The user defines the commercial and tax question; the agent decomposes that question into data structures, formulas, simulations, interfaces, verification steps, and documentation. The resulting sandbox makes tax assumptions visible and enables structured comparison without suggesting that an automated model can replace professional judgement.

Contents

CHAPTER 1

Introduction: Antigravity for Corporate Tax Power Users

This report introduces Antigravity for power users in corporate finance, international tax, transfer pricing, treasury, quantitative risk modeling, and tax technology. These professionals often face the same practical bottleneck: they can describe a sophisticated tax structure, but converting that structure into a working model, dashboard, and explanatory report usually requires a separate technical build. Antigravity narrows that gap. The case study is a multi-jurisdiction corporate tax planning application for a synthetic multinational conglomerate, Aether Global Group. The application lets a user compare domicile architectures, inspect intercompany payments, and see how tax rules change the global effective tax rate.

The central lesson is that Antigravity is not merely a code generator. It is an agentic development environment. The human user expresses intent, reviews the plan, redirects the design when needed, and evaluates the finished artifact. The agent translates that intent into files, simulations, mathematical engines, tests, and interface behaviour. For a corporate tax power user, this is valuable because the user can work at the level of tax logic—entity residence, transfer pricing, withholding tax treaties, financing structures, risk controls, and interpretation—while still obtaining source code that can be inspected, challenged, and extended.

The human interaction is therefore part of the system design. The user begins with a planning objective, observes what Antigravity proposes, corrects the direction, and asks for refinements: a more appealing dashboard, a compliance risk panel, a detailed ledger, and a portable Google Colab notebook. The final product is not a static memorandum. It is a browser-based sandbox that implements multiple tax rules, calculates effective tax rates, models intercompany flows, and explains tax consequences through interactive visualizations. The project progressed through three human-agent phases.

1.1 Pedagogical Orientation for Tax and Finance Users

A useful way to understand Antigravity is to compare it with a senior tax partner working with a technical associate team. The partner does not specify each JavaScript function or each CSS selector. The partner states the professional objective: build a tool that makes multi-jurisdiction corporate tax planning understandable. Antigravity then behaves like a technical associate who can plan the work, create files, run commands, interpret errors, and revise the application. The important difference is that the collaboration occurs through natural language and leaves visible artifacts in the workspace.

The human interaction has three layers. First, the user supplies the tax intention. In this project the intention was not simply to draw a chart, but to make a corporate tax optimization sandbox that finance users could inspect. Second, the user reviews intermediate outputs. If the first version is too static, the user can ask for sliders. If the tax consequences are hidden, the user can ask for a ledger. If the notebook is not portable, the user can request a self-contained version. Third, the user defines acceptance criteria. A useful tax sandbox must not merely run; it must communicate trade-offs, show risks, and help the user reason about structures.

This style of work is especially important in taxation because correctness is not only syntactic. A model can compile and still mislead if it omits withholding taxes, treaty assumptions, thin-capitalization rules, controlled foreign corporation overlays, or OECD Pillar Two effects. Antigravity accelerates implementation, but the human remains responsible for judgement. The strongest workflow is collaborative: the tax specialist supplies doctrine, constraints, and professional judgement; the agent supplies decomposition, coding, testing, iteration, and documentation.

1.2 Practical Prompting Principles

For a corporate tax professional, the key skill is iterative prompting. A good prompt does not need to specify every line of code, but it should identify the business objects that matter: subsidiaries, jurisdictions, industries, tax rates, intercompany payments, financing flows, transfer pricing logic, withholding tax matrices, and diagnostic outputs. The first prompt can be broad; later prompts should be more precise. In this project, the human guided Antigravity through three phases: a core tax request, an interactive dashboard, and a portable notebook implementation.

A practical prompt should also identify the intended audience. A model built for a tax director needs a different interface from a model built for a developer. The tax director needs effective tax rate summaries, red-flag compliance warnings, and intuitive flow maps. The developer needs formulas, code structure, and reproducibility. Antigravity can serve both audiences because it can generate the application, the notebook, and the explanatory report from the same underlying model.

1.3 Phase 1: Initial Core Request

The human operator initiated the project with the prompt:

> *"I need to have a tutorial of high end tax planning optimization for a company in multijurisdiction multi industry design a optimal architecture. You are allowed to generate a synthetic case. The objective is to have an app that highlights the impact of multiple decisions and architectures for domiciliation and intercompany connection."*

Antigravity interpreted the prompt as a request for a synthetic multinational group with multiple subsidiaries, industries, and intercompany relationships. The agent selected the Aether Global Group case study, defined entities in the United States, Germany, the United Kingdom, Ireland, Switzerland, Singapore, the Netherlands, and the Cayman Islands, and then constructed a mathematical engine for corporate income tax, withholding tax, financing, royalties, transfer pricing, GILTI, and Pillar Two.

1.4 Phase 2: Transition to Interactive Sandbox

After the initial engine was scoped, the user directed the agent to continue. Antigravity moved from tax logic to product design. It created an application layout, a visual design system, and an SVG network map to represent intercompany payments. It added sliders so that users could adjust royalty intensity, financing rates, transfer-pricing markups, and compliance rules in real time. This phase converted the tax model from a batch calculation into an interactive sandbox.

The lesson is that the user did not need to request every technical component one by one. The agent inferred that an effective corporate tax planning app needed controls, KPIs, charts, risk flags, and a ledger. The human retained control over product direction, while the agent handled the operational

decomposition.

1.5 Phase 3: Google Colab Portability

The user then requested a portable notebook implementation:

> *"Now based on this example, generate a colab notebook that allows to implement the model and visualizations... make sure the Colab notebook has a dashboard dynamic with the look and feel of the one generated."*

Antigravity responded by building a self-contained Jupyter notebook. This step matters because tax research often moves between local workstations, academic notebooks, internal review, and presentation environments. A notebook version allows the model to be executed, inspected, and taught without relying on the original browser project. The notebook also demonstrates that Antigravity can repurpose the same logic across delivery formats.

CHAPTER 2

How Antigravity Works: Execution and Platform Mechanics

To understand the engineering behind the corporate tax optimizer, it is useful to examine how Antigravity converts domain language into executable artifacts. The human operator did not manually write the tax engine, dashboard, verification script, notebook, and report. Instead, the agent interpreted the request, decomposed it into operational steps, and used tools to create and revise the workspace.

2.1 How the Agent Actually Converts Intent into Work

Antigravity works by turning a human request into a sequence of operational steps. The first step is interpretation. The agent reads the prompt and identifies the objects implied by it. In the phrase "multi-jurisdiction multi-industry tax planning optimization," the implied objects are legal entities, jurisdictions, tax rates, intercompany payments, treaty assumptions, interest limitations, transfer pricing markups, tax KPIs, compliance diagnostics, and a user interface. A conventional programming environment would require the user to define these explicitly before work begins. Antigravity can infer an initial structure, present it through a plan or implementation path, and then start building.

The second step is decomposition. The agent separates the project into files and responsibilities. A tax engine file should calculate entity-level taxable income and tax liabilities. A front-end file should synchronize controls and display results. A stylesheet should make the product appealing. A notebook compiler should package the logic for Colab. A report should explain the process and formulas. This decomposition is valuable because it keeps the project inspectable. A tax specialist can review the formulas without reading the entire interface layer.

The third step is tool execution. Antigravity is not limited to suggesting code in conversation. It can interact with the local workspace, write files, run commands, compile documents, and inspect outputs. When a command finishes, the result returns to the agent's context. If an error appears, the agent can revise its approach. This reactive loop is central. The agent proposes an action, the environment executes it, and the output becomes evidence for the next decision. In tax technology terms, this resembles a model validation loop: hypothesis, implementation, measurement, diagnosis, and revision.

The fourth step is review and customization. The user can interrupt the development trajectory at any time. If the application is technically correct but not useful, the user can say so. If the effective tax rate chart exists but does not explain the intercompany flow, the user can request a network diagram. If the report is too short or insufficiently pedagogical, the user can require deeper explanation. Antigravity treats these requests as new design constraints rather than as separate projects.

The fifth step is preservation of artifacts. The final product is not merely an answer in a chat window. It remains in the workspace as source code and documentation. This matters for auditability, reproducibility, and learning. A tax power user can inspect the formula for the Pillar Two top-up, examine how

thin-capitalization limits are applied, or ask another developer to extend the application. Antigravity therefore sits between natural-language ideation and professional software engineering.

2.2 Why the Execution Loop Matters in Corporate Tax

The execution loop matters because tax applications require both speed and discipline. A fast prototype is useful only if the user can inspect and correct it. Antigravity's loop gives the human repeated opportunities to intervene. The user can request a plan before implementation, ask for a narrower change, review modified files, compile the document, or run the application. Each cycle improves the artifact while preserving traceability.

In a tax department or advisory firm, this style of work can reduce the distance between domain experts and technical implementation. A transfer pricing professional may understand the policy but not the browser code. A developer may understand the interface but not the tax consequences. Antigravity helps bridge this gap by translating domain language into implementation steps while keeping the resulting code visible. The power user can therefore collaborate with the agent in the same way one might collaborate with a technical team: define requirements, review deliverables, and request revisions.

The loop also supports experimentation. A user can ask Antigravity to create a baseline version, then a version with more aggressive royalty routing, then a version with higher substance in the intellectual-property hub, then a version that turns on Pillar Two. The important point is not that the model proves an optimal tax structure. Rather, it gives the user a transparent laboratory for understanding the consequences of assumptions.

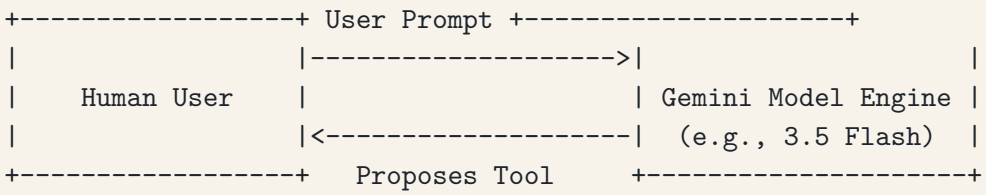
2.3 Unified Surfaces and TUI/GUI Channels

Antigravity can be understood as a family of surfaces for agentic development. In a graphical setting, the user interacts through a project workspace, editor panes, file previews, task panels, and browser verification. In a terminal setting, the user may interact through command-line tools and structured logs. In programmatic settings, agent instances can be orchestrated through SDK-style interfaces. The underlying pattern is the same: the human expresses intent, the agent proposes or performs operations, and artifacts accumulate in a visible workspace.

For corporate tax work, the graphical surface is especially valuable because the final product is visual. The user can inspect a dashboard, evaluate a flow diagram, and decide whether the interface communicates risk. The terminal and command surfaces remain important because they support compilation, verification, and reproducibility. A tax model that looks good but cannot be rerun is not sufficient for serious use.

2.4 The Reactive Agentic Execution Loop

Antigravity operates on a reactive event-driven cycle rather than a polling loop. The execution flow is modeled as follows:



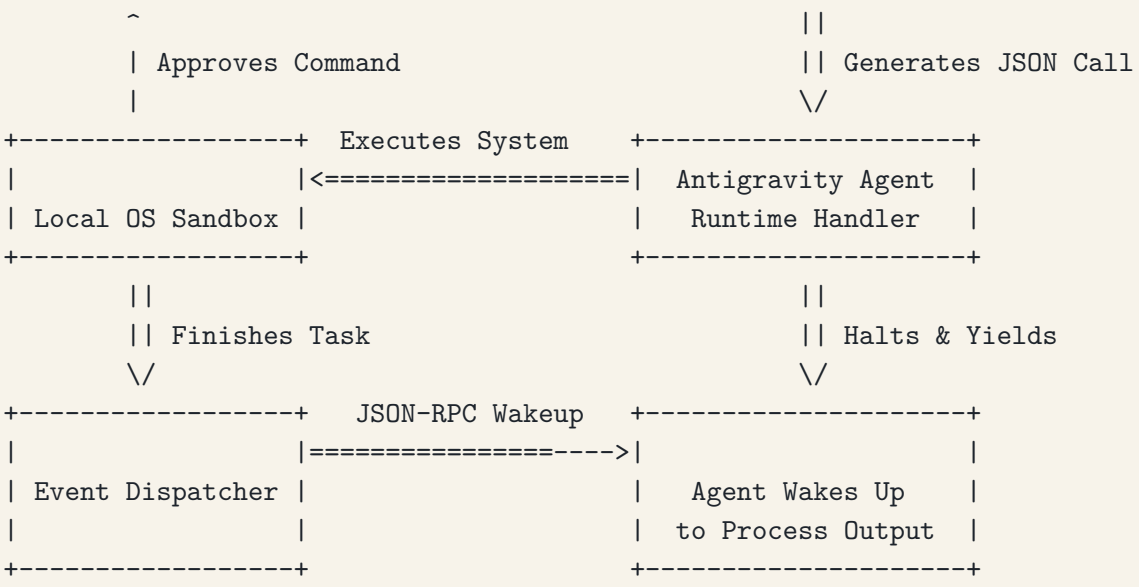


Figure 1: Reactive Agentic Loop Cycle

1. **Ingestion & Planning**: The agent receives user input, queries context pools and local rules, and plans its changes.
2. **Tool Call Proposal**: The agent issues a structured tool call, such as writing a file, running a command, or compiling a document.
3. **Halt and Yield**: The agent pauses while the environment executes the command or while the human reviews a proposed action.
4. **Reactive Event Wakeup**: When the command finishes or the user proceeds, the output returns to the agent and becomes evidence for the next step.

2.5 Isolation, Permissions, and Sandboxing

Security policies protect the host system during agent runs. This matters in tax and finance because models may contain confidential assumptions, client structures, and sensitive internal data. A disciplined agentic workflow therefore distinguishes between workspace files, external files, network access, and command execution.

- **Workspace Restrictions**: File actions are centered on the active project workspace so that changes remain localized and reviewable.
- **Command Policy Review**: Terminal commands are executed under policies that can require review, restrict actions, or run in safer environments.
- **Network Controls**: Network access can be constrained so that the agent does not exfiltrate data or rely on unapproved external sources.
- **Artifact Traceability**: Generated files, logs, and compiled outputs remain available for inspection after the interaction ends.

2.6 Skills & Customizations Architecture

Antigravity-style environments can also use local instructions, skills, and customization files. In a tax context this is important because institutions often have preferred modeling conventions. A firm may require that all tax rates be expressed as decimals, that all assumptions be surfaced in a separate

configuration file, or that all reports include limitations language. Skills and custom rules allow the agent to follow local practices without the user repeating them in every prompt.

This architecture is especially helpful for professionalization. A prototype created by a single user can become a repeatable workflow if the organization records its conventions. For example, a tax technology group might maintain a skill for transfer pricing dashboards, another for Pillar Two calculations, and another for LaTeX reporting. Antigravity then becomes not just a one-off assistant, but a reusable development interface.

CHAPTER 3

Agents Involved: Human Supervision and Multi-Agent Architecture

The development and verification of the corporate tax optimizer relied on a coordinated multi-agent structure. The human user acted as the product owner and domain expert: defining the tax objective, judging whether the interface was sufficiently appealing, and requesting changes. Antigravity acted as the implementation coordinator, turning those tax-oriented instructions into code, files, simulations, and tests.

3.1 Roles, Review, and Human Control

The human role in Antigravity is not passive. In a tax setting, this is essential because correctness is not only syntactic. A program can run without errors and still be financially misleading. For example, a tax model may omit withholding tax treaties, use outdated corporate tax rates, ignore local anti-avoidance rules, or present effective tax rates without explaining Pillar Two consequences. Antigravity can implement a request, but the human must evaluate whether the request itself is complete.

The agent's role is to reduce the cost of experimentation. A human can ask for a change that would normally require several technical steps: modify simulator state, change data structures, update chart rendering, alter layout, and document the new behaviour. Antigravity can perform those steps as one coherent development task. This allows power users to review more alternatives. Instead of accepting the first dashboard, the user can ask for a clearer ETR graph, a more attractive visual design, additional tax rules, or a different domiciliation strategy.

Human control also includes stopping the agent when the product direction is wrong. If the user wants a pedagogical tool rather than an aggressive tax avoidance optimizer, the user can specify that the model should highlight compliance risks and uncertainty. If the user wants an academic case study, the user can request synthetic data and avoid client-specific facts. This control is central to responsible use.

3.2 From Natural Language to Files

The Antigravity workflow converted natural language into concrete files. A high-level tax prompt became an application scaffold. The scaffold became a tax engine. The tax engine became a dashboard. The dashboard became a notebook. The notebook became a LaTeX report. At every step, the resulting artifacts were inspectable.

For a power user, this is a major advantage over a pure chat response. A chat response may explain how a tax model could be built, but it does not necessarily create the model. Antigravity creates files that can be opened, edited, compiled, and shared. This makes the interaction closer to a development engagement than to a question-answering session.

The file-based workflow also supports auditability. A user can compare versions, inspect formulas, or identify which assumptions drive the effective tax rate. This is particularly important for corporate tax planning because stakeholders may disagree about assumptions. The application does not eliminate judgement, but it makes the judgement visible.

3.3 Pedagogical Summary of the Agent Team

The agent team can be summarized as follows. The human supervisor defines purpose and quality. The core agent interprets the request and coordinates implementation. Background tasks run builds, checks, or long operations. Browser or visual verification loops inspect whether the product appears as intended. Documentation tasks explain what was built.

This structure mirrors a professional team. A tax partner defines the client problem. A manager decomposes the work. Associates build models and slides. Reviewers check quality. Antigravity compresses this structure into an interactive development environment while preserving the need for human judgement.

CHAPTER 4

Agent Workflow: From Human Prompt to Working Tax Application

This section explains how Antigravity moved from a high-level human prompt to a working corporate tax optimization application. The process is useful for tax and finance power users because it shows what kinds of instructions produce concrete outputs. A vague instruction such as "build a tax app" can start the process, but the most valuable refinements come when the human specifies the desired tax workflow: compare structures, show effective tax rates, calculate Pillar Two top-up, and expose compliance risks.

Antigravity's workflow combined planning, environment discovery, math construction, interface layout, notebook packaging, and validation. The agents took the following steps to build and verify the system:

1. **Case Design**: Defined Aether Global Group as a synthetic multinational with operating, financing, intellectual-property, and holding functions.
2. **Entity Mapping**: Assigned subsidiaries to jurisdictions including the United States, Germany, the United Kingdom, Ireland, Switzerland, Singapore, the Netherlands, and the Cayman Islands.
3. **Math Engine Construction**: Coded the simulation logic for corporate income tax, withholding taxes, thin-capitalization limits, GILTI, and Pillar Two.
4. **Dashboard Implementation**: Built a browser interface with KPI cards, sliders, an intercompany network visualization, a ledger table, charts, and risk diagnostics.
5. **Notebook Portability**: Generated a self-contained notebook version so the same tax model could be reviewed in a research or classroom environment.
6. **Documentation**: Produced this LaTeX report to explain the workflow, architecture, formulas, code, and lessons for power users.

4.1 What Power Users Should Learn from the Workflow

The first lesson is that prompts should define business meaning. Antigravity can infer a great deal, but it performs best when the user identifies the objects that matter. In corporate tax, those objects include entities, functions, risks, assets, payments, tax bases, and regulatory overlays. A prompt such as "show how IP routing affects Pillar Two and GILTI" gives the agent a much clearer target than "make it better."

The second lesson is that the user should review both results and assumptions. A dashboard can show a low effective tax rate, but the user must ask whether the assumptions are defensible. Are the royalties arm's length? Is the financing ratio sustainable? Does the low-tax entity have enough substance? Are withholding taxes correctly modeled? Antigravity can surface these questions through risk panels, but the human must decide how to interpret them.

The third lesson is that a good Antigravity workflow produces reusable artifacts. The application, notebook, and report are not separate deliverables; they are different views of the same model. The browser app supports exploration. The notebook supports research and teaching. The report supports

explanation and audit.

4.2 Review as a Design Discipline

Review is not merely proofreading. In an agentic workflow, review is product design. Each user response becomes a design constraint. If the user says the dashboard is not appealing, the agent must improve layout, color, typography, and interaction. If the user says the model is too abstract, the agent must add concrete ledgers and formulas. If the user says to be careful with margins, the agent must adjust the LaTeX source and compile the PDF.

For tax work, review should include four dimensions: technical correctness, tax plausibility, interpretability, and presentational quality. Technical correctness asks whether the code runs and the formulas are implemented consistently. Tax plausibility asks whether the assumptions reflect recognizable rules. Interpretability asks whether a non-developer can understand the output. Presentational quality asks whether the application and report are persuasive enough for a professional audience.

4.3 Customization During the Workflow

Customization is the difference between a generic demo and a useful professional prototype. In this project, customization occurred through the synthetic corporate group, the tax rules, the interactive sliders, the risk diagnostics, and the notebook packaging. Each customization made the product more relevant to corporate tax planning.

The same workflow could be customized for other tax problems. A user could replace the synthetic group with a mining group, a pharmaceutical group, a digital services group, or a banking group. The user could add controlled foreign corporation rules for another country, a treaty matrix for specific jurisdictions, or more detailed transfer pricing documentation. Antigravity is valuable because it can revise the same project rather than restarting from scratch.

CHAPTER 5

Iterative Development: Human Feedback as Product Design

The corporate tax optimizer improved through iteration. This is not incidental. Agentic development is most effective when the user treats each output as a draft. The first version establishes structure. The second version improves usability. The third version improves portability and explanation. Each version makes the product more useful.

5.1 Customization, Improvement, and Review Cycles

The review cycle in this project followed a simple pattern. The user described a goal. Antigravity built a version. The user evaluated the result. The agent revised the artifact. This cycle repeated across the browser app, notebook, and LaTeX report. The same pattern can be used in professional tax technology work.

The important point is that feedback should be specific. "Make it more appealing" can trigger visual improvements, but "add a compliance risk panel that explains which structures are fragile" produces a more targeted result. "Be careful with margins" is also useful because it gives the agent a concrete quality constraint for the PDF. The best human-agent collaboration combines broad intent with precise corrections.

5.2 Version 1.0: Static Tax Architecture Engine

The first version of the project can be understood as a static tax architecture engine. It defined the entities, stored jurisdictional assumptions, calculated taxable income, and applied basic corporate income tax. This version was essential because the visual dashboard needed a computational foundation.

A static engine is similar to a spreadsheet model. It can calculate outcomes, but it does not necessarily invite exploration. A user can inspect a result, but changing assumptions may require editing code or data. For a corporate tax power user, this is useful but incomplete. The model becomes much more powerful when assumptions become interactive controls.

5.3 Version 2.0: Interactive Intercompany Flow Sandbox

The second version transformed the model into an interactive sandbox. Sliders allowed users to adjust royalties, financing, markups, and rule toggles. The network view showed how payments moved between entities. KPI cards summarized global effective tax rate, tax savings, top-up tax, and compliance rating.

This version changed the character of the product. The user no longer had to ask the agent to rerun the model for every scenario. Instead, the user could explore directly. This is important pedagogically

because tax planning is comparative. A structure is meaningful only relative to alternatives. The sandbox made those alternatives visible.

5.4 Version 3.0: Portable Notebook and Documentation Layer

The third version added portability and explanation. The notebook allowed the model to be run outside the original browser environment. The LaTeX report explained the workflow, mathematical formulas, and code implementation. This layer is important because tax models are not only used; they are also reviewed, taught, and challenged.

A notebook is particularly useful for academic and training contexts. A professor can use it to demonstrate how changing a royalty rate affects tax outcomes. A student can inspect the code. A practitioner can adapt the assumptions. The report then provides the narrative that connects the model to the Antigravity workflow.

5.5 Why Iteration Produces a Better Tax Tool

Iteration produces a better tax tool because the first implementation rarely captures all requirements. Tax work is inherently conditional. Rules interact. Assumptions matter. Visual clarity matters. The user may discover new requirements only after seeing the first version.

Antigravity supports this process because it treats changes as part of the same project. The agent can revise files, preserve prior logic, add new components, and recompile outputs. The result is not just faster coding; it is faster convergence between domain intent and working software.

CHAPTER 6

Final Product: Appealing Corporate Tax Optimization Sandbox

The final product is a corporate tax optimization sandbox for a synthetic multinational group. It allows users to explore how legal entity structure, intercompany payments, financing, and regulatory overlays affect global tax outcomes. It is designed as a pedagogical and analytical tool, not as tax advice.

6.1 What Makes the Product Appealing and Useful

The product is appealing because it makes a complex topic visible. Corporate tax planning often lives in dense memoranda, spreadsheets, and legal diagrams. The Antigravity app converts those concepts into interactive controls and visual feedback. A user can change a royalty assumption and immediately see the effect on effective tax rate, top-up tax, and compliance risk.

The product is useful because it connects high-level strategy to mechanics. A user does not merely see that a structure is "efficient." The user can inspect the ledger, observe which entity receives income, see whether withholding tax applies, and identify whether Pillar Two neutralizes the benefit. This avoids a common weakness in tax models: presenting one optimized number without explaining the path that produced it.

The product is also useful because it is inspectable. The formulas are implemented in source code, summarized in the report, and portable to a notebook. This makes the sandbox suitable for teaching, internal experimentation, and early-stage prototyping.

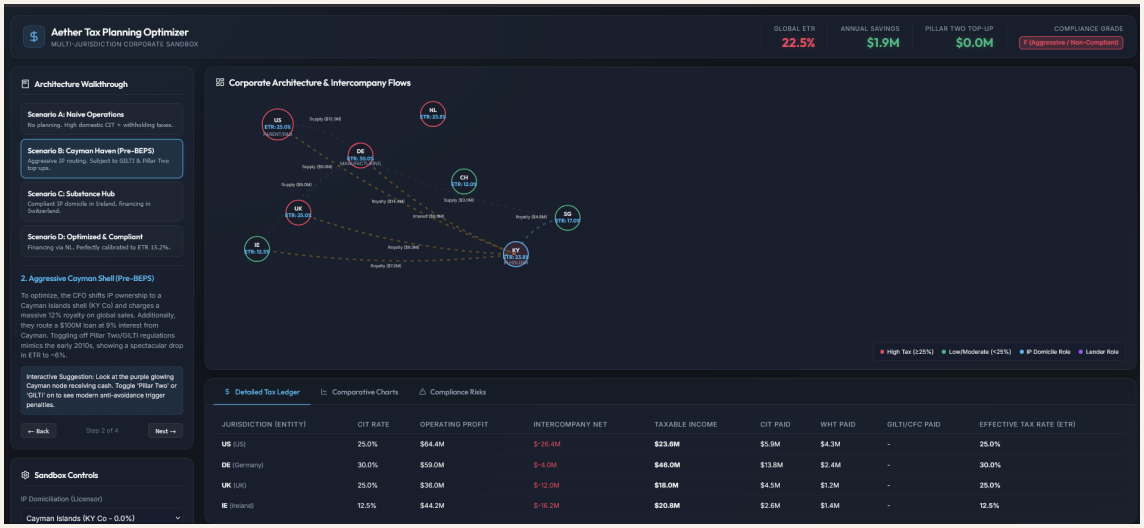


Figure 6.1. Printscreen of the Antigravity-generated corporate tax optimization sandbox used in this exercise.

6.2 Interpreting the Final Product as a Tax Power-User Template

The final product should be interpreted as a template for how tax power users can work with Antigravity. The specific jurisdictions and rates are synthetic and should not be treated as legal advice. The workflow, however, is transferable. A user can begin with a business problem, ask the agent to build a model, review the assumptions, request visual diagnostics, and preserve the result as code and documentation.

The template is especially relevant for areas where tax logic and computation overlap. Examples include transfer pricing scenario analysis, treasury financing structures, withholding tax planning, Pillar Two exposure analysis, supply-chain restructuring, and tax risk dashboards. In each case, the user can use Antigravity to turn conceptual rules into an inspectable prototype.

6.3 How Users Can Extend the Final Product

Users can extend the final product in several directions. They can add more jurisdictions, more detailed treaty rates, country-specific limitations, or scenario comparison tables. They can add stochastic assumptions, sensitivity analysis, or Monte Carlo simulations. They can connect the model to data files, export results, or create presentation-ready charts.

Developers can extend the code by separating assumptions from calculation logic, adding unit tests, or creating a formal schema for entities and payments. Tax professionals can extend the content by adding legal explanations, limitations, and review checklists. The important point is that the Antigravity workflow leaves the project in a state that can be continued.

6.4 Review Checklist for the Finished App

A finished tax optimization sandbox should be reviewed before it is used for decision support. The following checklist summarizes the most important review questions:

- **Assumptions**: Are jurisdictions, rates, payment types, and ownership relationships clearly stated?
- **Formulas**: Are taxable income, withholding tax, interest limitations, GILTI, and Pillar Two implemented consistently?
- **Interpretability**: Can a non-developer understand why the effective tax rate changes?
- **Risk**: Does the app flag aggressive or fragile structures rather than only showing savings?
- **Portability**: Can the model be rerun or inspected outside the original interaction?
- **Limitations**: Does the report make clear that the case is synthetic and not legal advice?

CHAPTER 7

Conclusion

The corporate tax optimization sandbox demonstrates the central value of Antigravity for professional users: it turns domain reasoning into inspectable software artifacts. The human user did not need to start with a blank codebase or specify every technical detail. Instead, the user supplied the tax objective, reviewed the output, and guided the product toward clearer analysis and presentation. Antigravity handled the operational work of decomposing the problem, writing files, building the interface, encoding formulas, and documenting the result.

The broader lesson is not that an agent can replace professional tax judgement. It cannot. Tax planning requires legal interpretation, commercial context, documentation, and risk assessment. The lesson is that Antigravity can make that judgement easier to test and communicate. By converting assumptions into a transparent model, it helps tax professionals compare structures, explain trade-offs, identify risk points, and teach complex rules through interactive examples.

For corporate tax power users, the most effective workflow is iterative. Start with the business question. Ask the agent for a model. Review the assumptions. Request visual diagnostics. Demand clear documentation. Compile and inspect the result. In that loop, Antigravity becomes less like a single-purpose automation tool and more like a collaborative development environment for turning expert reasoning into working analytical infrastructure.

References

1 OECD (2021). *Tax Challenges Arising from the Digitalisation of the Economy – Global Anti-Base Erosion Model Rules (Pillar Two)*. OECD Publishing, Paris.

2 Avi-Yonah, R. S. (2020). *International Tax as International Law: An Analysis of the International Tax Regime*. Cambridge University Press.

3 Gravelle, J. G. (2015). *Tax Havens: International Tax Avoidance and Evasion*. Congressional Research Service.

4 Financial Action Task Force (FATF). (2023). *Guidance on Beneficial Ownership for Legal Persons*. FATF, Paris.

5 United Nations (2021). *United Nations Model Double Taxation Convention between Developed and Developing Countries*. United Nations, New York.

APPENDIX A

Corporate Tax Model Formulations

This appendix provides mathematical and implementation details for the corporate tax optimization sandbox. The goal is not to reproduce a complete legal tax system. The goal is to show how Antigravity translated tax concepts into a transparent computational model that can be extended.

A.1 Entity-Level Taxable Income

For each entity i, let R_i^{local} represent local third-party revenue, E_i^{local} represent local operating expenses, RD_i represent research and development expense, RY_i^{in} and RY_i^{out} represent inbound and outbound royalties, I_i^{in} and I_i^{out} represent inbound and outbound interest, and TP_i^{in} and TP_i^{out} represent transfer-pricing service charges. The pre-adjustment taxable income is:

$$T_i = (R_i^{local} - E_i^{local} - RD_i) + RY_i^{in} + I_i^{in} + TP_i^{in} - RY_i^{out} - I_i^{out} - TP_i^{out}. \tag{A.1}$$

Corporate income tax before overlays is then:

$$CIT_i = \max(0, T_i) \times \tau_i, \tag{A.2}$$

where τ_i is the local corporate income tax rate.

A.2 Thin-Capitalization Interest Capping

Let $EBITDA_i$ be operating profit adjusted for intercompany royalties:

$$EBITDA_i = (R_i^{local} - E_i^{local} - RD_i) + RY_i^{in} - RY_i^{out}. \tag{A.3}$$

The allowable intercompany interest deduction is capped at 30% of EBITDA:

$$I_{allowable,i}^{out} = \max(0, EBITDA_i \times 0.30). \tag{A.4}$$

Any excess interest is added back to taxable income:

$$I_{excess,i} = \max(0, I_i^{out} - I_{allowable,i}^{out}), \quad T_i^{adjusted} = T_i + I_{excess,i}. \tag{A.5}$$

A.3 Withholding Tax Matrix

For a payment from entity a to entity b, let P_{ab} denote the payment amount and w_{ab} denote the treaty withholding rate. Withholding tax is:

$$WHT_{ab} = P_{ab} \times w_{ab}. \tag{A.6}$$

The global tax total includes both corporate income taxes and withholding taxes, while entity-level covered taxes may include withholding suffered depending on the reporting convention.

A.4 US GILTI Rules

For a controlled foreign corporation cfc owned by the US parent, let $Assets_{cfc}$ be tangible assets. The Qualified Business Asset Investment deemed return is:

$$QBAI_{cfc} = Assets_{cfc} \times 0.10. \tag{A.7}$$

The simplified intangible income is:

$$Intangible_{cfc} = \max(0, T_{cfc} - QBAI_{cfc}). \tag{A.8}$$

If the local effective tax rate is below the threshold used in the model, the US parent records a simplified GILTI charge:

$$GILTI_{tax} = \max(0, Intangible_{cfc} \times 50\% \times 21\% - CIT_{cfc} \times 80\%). \tag{A.9}$$

A.5 OECD Pillar Two Global Minimum Tax

If the effective tax rate in a jurisdiction falls below 15%, a top-up tax applies. Let covered taxes be:

$$CoveredTaxes_i = CIT_i + WHT_{suffered,i}. \tag{A.10}$$

The local effective tax rate is:

$$ETR_{local,i} = \frac{CoveredTaxes_i}{T_i}. \tag{A.11}$$

The top-up tax rate is:

$$\tau_i^{topup} = \max(0, 0.15 - ETR_{local,i}). \tag{A.12}$$

The substance-based income exclusion is:

$$SBIE_i = 5\% \times Payroll_i + 5\% \times Assets_i. \tag{A.13}$$

The top-up tax is:

$$TopUp_i = \tau_i^{topup} \times \max(0, T_i - SBIE_i). \tag{A.14}$$

APPENDIX B

JavaScript Implementation Examples

The following snippets show how the mathematical model can be implemented in JavaScript. They are included for developers who want to understand how Antigravity translated the tax model into executable application logic.

B.1 Pillar Two Minimum Tax Calculation

```
const pillarTwoReports = [];
if (rulePillar2) {
    for (const [code, ent] of Object.entries(entities)) {
        let coveredTaxes = ent.citPaid;
        const whtSuffered = whtDetails
            .filter(d => d.to === code)
            .reduce((sum, d) => sum + d.amount, 0);

        coveredTaxes += whtSuffered;
        const p2Income = ent.localTaxableIncome;
        const localEtr = p2Income > 0 ? coveredTaxes / p2Income : ent.citRate;
        ent.taxRateBeforePillar2 = localEtr;

        if (p2Income > 0 && localEtr < 0.15) {
            const topUpTaxRate = 0.15 - localEtr;
            const payrollCarveout = ent.payroll * 0.05;
            const assetCarveout = ent.tangibleAssets * 0.05;
            const sbie = payrollCarveout + assetCarveout;
            const excessProfits = Math.max(0, p2Income - sbie);
            const topUpTax = excessProfits * topUpTaxRate;

            if (topUpTax > 0) {
                ent.pillarTwoTopUpPaid = topUpTax;
                if (code === "KY") {
                    entities.US.pillarTwoTopUpPaid += topUpTax;
                } else {
                    ent.citPaid += topUpTax;
                }
            }
        }
    }
}
```

Listing B.1. Pillar Two minimum tax implementation

B.2 Simulation Update Loop

```
function updateSimulation() {
```

```
    const data = engine.simulate(state);

    kpiEtr.textContent = (data.summary.globalETR * 100).toFixed(1) + "%";
    kpiSavings.textContent = "$" +
        (data.summary.globalTaxSavings / 1000000).toFixed(1) + "M";

    const totalTopup = data.pillarTwoReports
        .reduce((sum, report) => sum + report.topUpTax, 0);
    kpiTopup.textContent = totalTopup > 0
        ? "$" + (totalTopup / 1000000).toFixed(2) + "M"
        : "$0.0M";

    kpiCompliance.innerHTML =
        `<span class="badge ${data.summary.complianceRating.class}">` +
        `${data.summary.complianceRating.grade}</span>`;

    renderSVGNetwork(data);
    renderLedgerTable(data);
    renderRisksList(data);

    if (state.activeTab === "charts") {
        drawCharts(data);
    }

    window.currentTaxData = data;
}
```

Listing B.2. State synchronization and dashboard update loop

APPENDIX C

Prompting Trail Used in this Example

This appendix records the prompting sequence used to create the Antigravity tax application and this accompanying LaTeX report. It is included so that tax and finance power users can see how the final product emerged through iterative human-agent interaction.

C.1 Prompt 1: Initial Tax Request

> *"I need to have a tutorial of high end tax planning optimization for a company in multijurisdiction multi industry design a optimal architecture. You are allowed to generate a synthetic case. The objective is to have an app that highlights the impact of multiple decisions and architectures for domiciliation and intercompany connection."*

This prompt established the project boundaries. The agent selected the corporate subsidiaries, defined the initial baseline operating data, and created the core math engine.

C.2 Prompt 2: Visual Dashboard and Sandbox Sliders

> *"OK"*

This prompt triggered the execution of the full implementation plan. The agent created the design system, implemented the SVG flow network, and wired up sliders for intercompany decisions.

C.3 Prompt 3: Google Colab Portability

> *"Now based on this example, generate a colab notebook that allows to implement the model and visualizations... make sure the Colab notebook has a dashboard dynamic with the look and feel of the one generated."*

This prompt directed the agent to build the notebook. To ensure portability, the agent inlined the dashboard logic and made the notebook self-contained.